

Lecture 5: Data Manipulation and Analysis (I) – Base R

Harris Coding Camp – Standard Track

Summer 2024

Data Analysis via Tidyverse and Base R

Tidyverse has become the leading way people clean & manipulate data in R

- ▶ These packages make certain data analysis easier than core Base R commands
 - ▶ Tidyverse commands can be more efficient (less lines of code)
-

Data Analysis via Tidyverse and Base R

However, you will inevitably run into edge cases where tidyverse commands don't work the way you expect them to, or where you have to reuse/debug codes written in Base R.

Also, many existing R scripts and functions are written in Base R, and familiarity with Base R is necessary to work with or modify codes.

Hence, it's still important to learn **Base R**. It's good to have a basic foundation on both approaches and then decide which you prefer when conducting data analysis!

Today's Lecture

Today, we will cover:

- ▶ subset() or subsetting operators to keep data that meet the criteria in Base R
 - ▶ subsetting operators: `[]`, `$`

Selecting columns and rows with `subset()`

Selecting columns and rows with subset()

Subset Variables (Columns)



Subset Observations (Rows)



Selecting columns and rows with subset()

The `subset()` is a Base R function which “select” observations (**rows**) and/or variables (**columns**).

Syntax: `subset(data_name, condition, select)`

- ▶ `data_name` indicates which dataset to be subset
- ▶ `condition` indicates which **rows** to keep
(if argument is not used, default will keep all rows)
- ▶ `select` indicates which **columns** to keep
(if argument is not used, default will keep all columns)

Investigating data

It is always suggested to take a look at the data before we move forward:

```
head(housing_data)
```

```
#> # A tibble: 6 x 9
```

```
#>   city      year month sales   volume median listings inventory date
#>   <chr>    <dbl> <dbl> <dbl>    <dbl> <dbl>    <dbl>    <dbl> <dbl>
#> 1 Abilene  2000     1    72  5380000  71400     701      6.3 2000
#> 2 Abilene  2000     2    98  6505000  58700     746      6.6 2000
#> 3 Abilene  2000     3   130  9285000  58100     784      6.8 2000
#> 4 Abilene  2000     4    98  9730000  68600     785      6.9 2000
#> 5 Abilene  2000     5   141 10590000  67300     794      6.8 2000
#> 6 Abilene  2000     6   156 13910000  66900     780      6.6 2000
```


Selecting columns with subset()

Show the data which only includes city, sales and listings:

```
subset(housing_data, select = c(city, sales, listings))
```

```
#> # A tibble: 8,602 x 3
```

```
#>   city      sales listings
```

```
#>   <chr>    <dbl>    <dbl>
```

```
#> 1 Abilene      72      701
```

```
#> 2 Abilene      98      746
```

```
#> 3 Abilene     130      784
```

```
#> 4 Abilene      98      785
```

```
#> 5 Abilene     141      794
```

```
#> 6 Abilene     156      780
```

```
#> 7 Abilene     152      742
```

```
#> 8 Abilene     131      765
```

```
#> 9 Abilene     104      771
```

```
#> 10 Abilene     101      764
```

```
#> # i 8,592 more rows
```

Selecting columns with subset()

Show the data which excludes city, sales and listings:

```
subset(housing_data, select = -c(city, sales, listings))
```

```
#> # A tibble: 8,602 x 6
```

```
#>   year month  volume median inventory date
```

```
#>   <dbl> <dbl>   <dbl> <dbl>      <dbl> <dbl>
```

```
#> 1  2000     1 5380000  71400      6.3 2000
```

```
#> 2  2000     2 6505000  58700      6.6 2000.
```

```
#> 3  2000     3 9285000  58100      6.8 2000.
```

```
#> 4  2000     4 9730000  68600      6.9 2000.
```

```
#> 5  2000     5 10590000 67300      6.8 2000.
```

```
#> 6  2000     6 13910000 66900      6.6 2000.
```

```
#> 7  2000     7 12635000 73500      6.2 2000.
```

```
#> 8  2000     8 10710000 75000      6.4 2001.
```

```
#> 9  2000     9  7615000 64500      6.5 2001.
```

```
#> 10 2000    10  7040000 59300      6.6 2001.
```

```
#> # i 8,592 more rows
```

Selecting rows and columns with subset()

Show the data focusing on city, sales and listings in Bay Area which (1) sales is at least 100; (2) listings exceeds 500:

```
subset(housing_data,  
       city == "Bay Area" &  
       sales >= 100 &  
       listings > 500,  
       select = c(city, sales, listings))
```

```
#> # A tibble: 186 x 3  
#>   city      sales listings  
#>   <chr>    <dbl>    <dbl>  
#> 1 Bay Area    244      1766  
#> 2 Bay Area    375      1817  
#> 3 Bay Area    391      1830  
#> 4 Bay Area    421      1845  
#> 5 Bay Area    533      1827  
#> 6 Bay Area    561      1874  
#> 7 Bay Area    449      1876  
#> 8 Bay Area    471      1869  
#> 9 Bay Area    397      1842  
#> 10 Bay Area   376      1904  
#> # i 176 more rows
```

Selecting rows and columns with subset()

Show the data focusing on city, sales and listings in Abilene **and** Bay Area which (1) sales is at least 100; (2) listings exceeds 500:

```
subset(housing_data,  
       (city == "Abilene" & city == "Bay Area") &  
       sales >= 100 &  
       listings > 500,  
       select = c(city, sales, listings))  
#> # A tibble: 0 x 3  
#> # i 3 variables: city <chr>, sales <dbl>, listings <dbl>
```

No rows are shown here! What's wrong with our code?

Aside: AND in English v. in R

Notice the different definitions of **AND** between R and plain English!

- ▶ In English, **AND** is a conjunction
- ▶ In R, **AND (&)** is a logical operator!
 - ▶ TRUE & TRUE yields TRUE $\Rightarrow$ only observations which satisfy those conditions will be kept
 - ▶ Otherwise, R yields FALSE $\Rightarrow$ observations which do not satisfy one (or more) conditions will not be kept

```
subset(housing_data,  
  (city == "Abilene" & city == "Bay Area") &  
  sales >= 100 &  
  listings > 500,  
  select = c(city, sales, listings))  
#> # A tibble: 0 x 3  
#> # i 3 variables: city <chr>, sales <dbl>, listings <dbl>
```

There is no observation whose city is recorded in **both** Abilene and Bay Area in our dataset! So, how do we fix it?

Selecting rows and columns with subset()

In R, we need to use OR (|) operator instead.

- ▶ R yields TRUE if there is at least one condition is TRUE
- ▶ That is, observations which satisfy at least one condition will be kept

```
subset(housing_data,  
       (city == "Abilene" | city == "Bay Area") &  
       sales >= 100 &  
       listings > 500,  
       select = c(city, sales, listings))
```

```
#> # A tibble: 352 x 3
```

```
#>   city      sales listings
```

```
#>   <chr>    <dbl>    <dbl>
```

```
#> 1 Abilene    130      784
```

```
#> 2 Abilene    141      794
```

```
#> 3 Abilene    156      780
```

```
#> 4 Abilene    152      742
```

```
#> 5 Abilene    131      765
```

```
#> 6 Abilene    104      771
```

```
#> 7 Abilene    101      764
```

```
#> 8 Abilene    100      721
```

```
#> 9 Abilene    112      700
```

```
#> 10 Abilene   118      738
```

Selecting rows and columns with subset()

We can also use %in% operator in place of OR operator (|):

```
subset(housing_data,  
       city %in% c("Abilene", "Bay Area") &  
       sales >= 100 &  
       listings > 500,  
       select = c(city, sales, listings))
```

```
#> # A tibble: 352 x 3  
#>   city      sales listings  
#>   <chr>    <dbl>    <dbl>  
#> 1 Abilene    130      784  
#> 2 Abilene    141      794  
#> 3 Abilene    156      780  
#> 4 Abilene    152      742  
#> 5 Abilene    131      765  
#> 6 Abilene    104      771  
#> 7 Abilene    101      764  
#> 8 Abilene    100      721  
#> 9 Abilene    112      700  
#> 10 Abilene   118      738  
#> # i 342 more rows
```

Selecting rows and columns with subset()

Now, let's **create a new data frame** focusing on city, sales and listings in Abilene and Bay Area

- Combine subset() with <- to create a new data frame

```
new_housing_data <- subset(housing_data,  
                           city %in% c("Abilene", "Bay Area"),  
                           select = c(city, sales, listings))
```

Let's take a look at the new dataset to see if our code works correctly:

```
head(new_housing_data)  
#> # A tibble: 6 x 3  
#>   city    sales listings  
#>   <chr>   <dbl>   <dbl>  
#> 1 Abilene    72     701  
#> 2 Abilene    98     746  
#> 3 Abilene   130     784  
#> 4 Abilene    98     785  
#> 5 Abilene   141     794  
#> 6 Abilene   156     780
```


Try it yourself – Use `subset()`

1. Create a new data frame by extracting the columns `city`, `year`, `month`, `volume` and `median` from `texas_housing_data`.
2. Create a new data frame from `texas_housing_data` that includes observations with `listings` more than 5500 and `median` no less than 235000. Count the number of the selected observations.
3. Count the number of observations from Arlington, Bay Area and Collin County with `listings` less than 3000.
4. Subset all observations from Arlington, Bay Area and Collin County with `listings` at least 3500, and only keep variables `city`, `year`, `month` and `listings` in your new data frame.

Subsetting using subsetting operators

Subsetting to extract elements

Subsetting operators can be used to select or exclude columns and/or rows

- ▶ subsetting operators: `[]`, `$`
- ▶ these operators function differently based on data types (e.g, vectors, lists, data frames)

Review: Vector vs List vs Data Frame

The diagram shows a table with 10 rows and 5 columns. A red box labeled 'Data Frame' encompasses the entire table. A green box labeled 'Vector' highlights the 'year' column. A blue box labeled 'List' highlights the third row of the table.

	country	year	strike.volume	unemployment
1	Australia	1951	296	1.3
2	Australia	1952	397	2.2
3	Australia	1953	360	2.5
4	Australia	1954	3	1.7
5	Australia	1955	326	1.4
6	Australia	1956	352	1.8
7	Australia	1957	195	2.3
8	Australia	1958	133	2.7
9	Australia	1959	109	2.6
10	Australia	1960	208	2.5

- ▶ Elements of a vector/column must be the same type
- ▶ Elements of a list/row can be different types

Subsetting elements of vectors using `[]`

Subsetting a vector refers to isolating particular elements of a vector. Three ways to subset a vector using `[]`:

1. Using **positive integers** to return elements at specified positions
2. Using **negative integers** to exclude elements at specified positions
3. Using **conditions** to return elements where conditions hold

Subsetting elements of vectors using []

1. Using **positive integers** to return elements at specified positions

```
# Create numeric vector `x`  
x <- c(21, 22, 23, 24, 25)
```

```
# Return 5th element  
x[5]  
#> [1] 25
```

```
# Return 2nd and 4th elements  
x[c(2, 4)]  
#> [1] 22 24
```

```
# Return 3rd through 5th elements  
x[3:5]  
#> [1] 23 24 25
```

Subsetting elements of vectors using []

We can reassign accessed values too.

```
x[3:5]
```

```
#> [1] 23 24 25
```

```
# reassign the 5th element
```

```
x[5] <- 30
```

```
x[3:5]
```

```
#> [1] 23 24 30
```

Subsetting elements of vectors using []

2. Using **negative integers** to exclude elements at specified positions

```
x  
#> [1] 21 22 23 24 30
```

```
# exclude 1st element
```

```
x[-1]  
#> [1] 22 23 24 30
```

```
# exclude 2nd and 3rd elements
```

```
x[-c(2, 3)]  
#> [1] 21 24 30
```


Subsetting elements of vectors using []

3. Using **conditions** to return elements where conditions hold

```
x  
#> [1] 21 22 23 24 30
```

```
# return all elements of `x` where element is greater than 23  
x[x > 23]  
#> [1] 24 30
```

```
# return all elements of `x` where element is no greater than 22  
x[x <= 22]  
#> [1] 21 22
```

```
# Replace elements that meet the condition with 0  
x[x <= 22] <- 0  
x  
#> [1] 0 0 23 24 30
```

Subsetting elements of lists using []

Using [] to subset a list works the same as subsetting a vector:

```
list_example <- list(year = 2024, month = 9, date = 10, dept = "harris")  
str(list_example)
```

```
#> List of 4  
#> $ year : num 2024  
#> $ month: num 9  
#> $ date : num 10  
#> $ dept : chr "harris"
```

create a new list that contains 4th and 1st elements of old list

```
list_new <- list_example[c(4, 1)]
```

```
str(list_new)
```

```
#> List of 2  
#> $ dept: chr "harris"  
#> $ year: num 2024
```

Subsetting elements of data frames using []

Use **double** indices to extract *rows and columns* based on row & column number:

```
# extract rows 2-3 and columns 1-5
housing_data[2:3, 1:5]
#> # A tibble: 2 x 5
#>   city      year month sales  volume
#>   <chr>    <dbl> <dbl> <dbl>   <dbl>
#> 1 Abilene  2000      2    98 6505000
#> 2 Abilene  2000      3   130 9285000
```

```
# extract rows 101-103 and columns 1, 2, 3 and 6
housing_data[101:103, c(1, 2, 3, 6)]
#> # A tibble: 3 x 4
#>   city      year month median
#>   <chr>    <dbl> <dbl>   <dbl>
#> 1 Abilene  2008      5 104000
#> 2 Abilene  2008      6 110400
#> 3 Abilene  2008      7 106400
```

```
# This works the same
housing_data[101:103, c("city", "year", "month", "median")]
```

Subsetting elements of data frames using []

Use **double** indices to extract *rows and columns* based on row & column number:

```
# return specific rows and all columns
```

```
housing_data[101:103, ]
```

```
#> # A tibble: 3 x 9
```

```
#>   city      year month sales  volume median listings inventory date
#>   <chr>    <dbl> <dbl> <dbl>    <dbl>  <dbl>    <dbl>    <dbl> <dbl>
#> 1 Abilene  2008     5   182 21314933 104000     951      6.3 2008.
#> 2 Abilene  2008     6   180 21146068 110400    1000      6.7 2008.
#> 3 Abilene  2008     7   171 19502253 106400    1013      7.1 2008.
```

```
# return all rows and specific columns
```

```
housing_data[, c(1, 2, 3, 6)]
```

```
#> # A tibble: 8,602 x 4
```

```
#>   city      year month median
#>   <chr>    <dbl> <dbl>  <dbl>
#> 1 Abilene  2000     1   71400
#> 2 Abilene  2000     2   58700
#> 3 Abilene  2000     3   58100
#> 4 Abilene  2000     4   68600
#> 5 Abilene  2000     5   67300
#> 6 Abilene  2000     6   66900
#> 7 Abilene  2000     7   73500
```

Subsetting data frames using `[]` + `$`

We can also use **conditions** to subset the data.

Syntax: `data_name[<condition to keep rows>, <select columns>]`

- Need to use **double** indices

```
# All rows recorded in the year 2010 or later & listings > 1000  
# All columns  
housing_data[housing_data$year >= 2010 &  
              housing_data$listings > 1000, ]
```

```
# All rows recorded in the year 2010 but only the first 3 columns  
housing_data[housing_data$year == 2010, 1:3]
```

Summary: `[] + $` v. `subset()`

We can use either `[] + $` or `subset()` to subset data frames:

```
# Extract specific rows and specific columns
```

```
# Method 1
```

```
housing_data[housing_data$year == 2010, 1:3]
```

```
# Method 2
```

```
subset(housing_data, year == 2010, select = c(1:3))
```

Try it yourself – Use subsetting operators

Instead of using `subset()`, try to use the subsetting operators (`[]`, `$`) to redo the questions:

1. Create a new data frame by extracting the columns `city`, `year`, `month`, `volume` and `median` from `housing_data`.
2. Create a new data frame from `housing_data` that includes observations with `listings` more than 5500 and `median` no less than 235000. Count the number of the selected observations.
3. Subset all observations from Arlington, Bay Area and Collin County with `listings` at least 3500, and only keep variables `city`, `year`, `month` and `listings` in your new data frame.

Recap

We learned:

- ▶ Quick comparison between Tidyverse and Base R
- ▶ `subset()` or subsetting operators to keep data that meet the criteria in base R
 - ▶ subsetting operators: `[]`, `$`
 - ▶ these operators function differently based on data types (e.g, vectors, lists, data frames)